

DAY 11 — SUBQUERIES

Today we learn **Subqueries**.

You've already learned:

Day 1–7 → single-table SQL

Day 8 → INNER JOIN

Day 9 → LEFT / RIGHT / FULL JOIN

Day 10 → Multiple JOINS

Now we'll learn how to put **one SQL query inside another SQL query**.

Subqueries are extremely useful for:

- comparing values against averages
- finding records based on another query
- filtering using aggregated results
- existence checks
- building analytical queries
- RAG/document analytics

1. What Is a Subquery?

A **subquery** is a SQL query inside another SQL query.

Example:

```
SELECT
title,
page_count
FROM documents
WHERE page_count > (
    SELECT AVG(page_count)
    FROM documents
);
```

There are two queries.

Outer query

```
SELECT
```

```
title,  
  
page_count  
  
FROM documents  
WHERE page_count > (...);
```

Inner query

```
SELECT AVG(page_count)  
FROM documents;
```

The inner query produces a value.

Then the outer query uses that value.

Conceptually:

Inner Query

↓

AVG(page_count)

↓

Outer Query

↓

documents above average

2. Why Do We Need Subqueries?

Suppose the average document size is:

35 pages

You want:

Documents larger than the average.

You don't want to manually write:

```
WHERE page_count > 35
```

because the average could change.

Instead:

```
WHERE page_count > (  
    SELECT AVG(page_count)  
    FROM documents  
)
```

The database calculates the comparison dynamically.

3. Basic Subquery Structure

General pattern:

```
SELECT ...  
FROM ...  
WHERE column operator (  
    SELECT ...  
    FROM ...  
);
```

For example:

```
SELECT  
title,  
  
page_count  
  
FROM documents  
WHERE page_count > (  
    SELECT AVG(page_count)  
    FROM documents  
);
```

The inner query must produce something compatible with the operator.

Here:

>

expects a single comparable value.

The subquery returns:

AVG(page_count)
which is one number.

4. Scalar Subquery

A subquery returning **one value** is often called a scalar subquery.

Example:

```
SELECT AVG(page_count)  
FROM documents;
```

Result:

35.4

```
Then:
SELECT
title,

page_count

FROM documents
WHERE page_count > (
    SELECT AVG(page_count)
    FROM documents
);
```

The outer query compares every document against that one number.

5. Another Example: Maximum

Find documents having the maximum page count.

```
SELECT
title,

page_count

FROM documents
WHERE page_count = (
    SELECT MAX(page_count)
    FROM documents
);
```

Notice this is better than:

```
ORDER BY page_count DESC
LIMIT 1;
```

if you want **all documents tied for maximum**.

Example:

Document A → 100 pages

Document B → 100 pages

The subquery returns:

100

Both documents match.

6. Subquery With MIN

Find documents smaller than the average:

```
SELECT
title,

page_count

FROM documents
WHERE page_count < (
    SELECT AVG(page_count)
    FROM documents
);
```

7. Subquery With SUM

Suppose:

Find users whose total document pages exceed 100.

First, we need to calculate total pages per user.

```
SELECT
user_id,

    SUM(page_count) AS total_pages
FROM documents
GROUP BY user_id;
```

But now suppose we want users whose total is greater than the **average user total**.

This becomes more interesting.

We can use a subquery:

```
SELECT
user_id,

    SUM(page_count) AS total_pages
FROM documents
GROUP BY user_id
HAVING SUM(page_count) > (
    SELECT AVG(total_pages)
    FROM (
        SELECT
            user_id,

            SUM(page_count) AS total_pages
        FROM documents
        GROUP BY user_id
```

```
    ) user_totals  
  );
```

Don't worry if this looks difficult.

We're nesting queries:

documents

↓

GROUP BY user

↓

user totals

↓

average user total

↓

compare each user's total

This is an important advanced SQL pattern.

8. Subquery in WHERE

This is the most common type you'll learn today.

Example:

Find processed documents that are larger than the average document.

```
SELECT  
title,  
  
page_count  
  
FROM documents  
WHERE status = 'processed'  
AND page_count > (  
    SELECT AVG(page_count)  
    FROM documents  
);
```

Important:

The subquery calculates the average over **all documents**, not only processed documents.

If the requirement were:

Find processed documents larger than the average processed document

then the subquery needs:

```
SELECT AVG(page_count)
FROM documents
WHERE status = 'processed';
```

So:

```
SELECT
title,

page_count

FROM documents
WHERE status = 'processed'
AND page_count > (
    SELECT AVG(page_count)
    FROM documents
    WHERE status = 'processed'
);
```

This distinction is very important.

9. Subquery + IN

Sometimes a subquery returns **multiple rows**.

Example:

Find documents owned by users from India.

First:

```
SELECT id
FROM users
WHERE country = 'India';
```

Potential result:

1

4

Now:

```
SELECT
title,

user_id

FROM documents
WHERE user_id IN (
```

```
SELECT id
FROM users
WHERE country = 'India'
);
```

Here the subquery returns:

1

4

That's multiple values.

Therefore:

IN

is appropriate.

10. Why = Can Fail Here

This is wrong if the subquery returns multiple rows:

```
SELECT
title

FROM documents
WHERE user_id = (
    SELECT id
    FROM users
    WHERE country = 'India'
);
```

Suppose India has:

1

4

Then SQL effectively gets:
user_id = (1, 4)

But = expects one value.

Use:

IN

instead.

11. IN With RAG Data

Suppose:

Find documents owned by active users.

```
SELECT
id,

title,

user_id

FROM documents
WHERE user_id IN (
    SELECT id
    FROM users
    WHERE account_status = 'active'
);
```

This is a perfectly valid alternative to a JOIN.

Equivalent JOIN:

```
SELECT
d.id,

d.title,

d.user_id

FROM documents d
JOIN users u
    ON d.user_id = u.id
WHERE u.account_status = 'active';
```

Both can solve the problem.

12. JOIN vs Subquery

This is an important engineering question.

```
JOIN
SELECT
d.title

FROM documents d
JOIN users u
    ON d.user_id = u.id
WHERE u.country = 'India';
```

Subquery

```
SELECT
title

FROM documents
WHERE user_id IN (
  SELECT id
  FROM users
  WHERE country = 'India'
);
```

Both are valid.

The choice depends on:

- readability
- query structure
- whether you need columns from the other table
- optimizer behavior
- data volume
- business logic

Don't assume:

"JOIN is always faster."

Or:

"Subquery is always slower."

Modern PostgreSQL can optimize many equivalent query forms.

13. Subquery With NOT IN

Example:

Find users who don't own any documents.

```
SELECT
id,
name

FROM users
WHERE id NOT IN (
  SELECT user_id
  FROM documents
);
```

This looks simple.

But there's an important SQL problem involving NULL.

14. The NOT IN + NULL Trap

Suppose:

documents.user_id

contains:

1

2

NULL

Then:

WHERE id NOT IN (

1,

2,

NULL

)

can produce unexpected results because SQL uses three-valued logic:

TRUE

FALSE

UNKNOWN

This is one reason experienced SQL developers often prefer:

NOT EXISTS

for this type of problem.

We'll learn EXISTS shortly.

15. EXISTS

EXISTS asks:

Does at least one matching row exist?

Example:

Find users who have at least one document.

```
SELECT
  u.id,
  u.name
FROM users u
WHERE EXISTS (
  SELECT 1
  FROM documents d
  WHERE d.user_id = u.id
);
```

The subquery is **correlated** with the outer query.

Look at:

```
WHERE d.user_id = u.id
```

The inner query refers to:

u.id

from the outer query.

16. How EXISTS Works

For each user, PostgreSQL conceptually asks:

Does a document exist

where document.user_id = this user's id?

For Aarav:

Does document.user_id = 1 exist?

Yes → keep Aarav.

For Maya:

Does document.user_id = 7 exist?

No → remove Maya.

17. NOT EXISTS

Now:

Find users who have no documents.

```
SELECT
  u.id,

  u.name

FROM users u
WHERE NOT EXISTS (
  SELECT 1
  FROM documents d
  WHERE d.user_id = u.id
);
```

This is often a very clean way to express:

Find parent records without child records.

18. EXISTS vs IN

These can express similar logic.

IN

```
SELECT
  id,

  name

FROM users
WHERE id IN (
  SELECT user_id
  FROM documents
);
```

EXISTS

```
SELECT
  u.id,

  u.name

FROM users u
WHERE EXISTS (
  SELECT 1
  FROM documents d
```

```
WHERE d.user_id = u.id  
);
```

The conceptual difference:

IN

→ Is my value in this set?

EXISTS

→ Does a matching row exist?

19. Why SELECT 1?

You may see:

```
SELECT 1  
FROM documents d  
WHERE d.user_id = u.id
```

inside EXISTS.

Why 1?

Because EXISTS doesn't care what the subquery returns.

It only cares:

row exists?

So:

```
SELECT 1
```

is conventional.

This would also work:

```
SELECT d.id
```

But SELECT 1 clearly communicates the intent.

20. Correlated vs Non-Correlated Subqueries

This is a key Day 11 concept.

Non-correlated

The subquery doesn't depend on the outer query.

```
SELECT
title,

page_count

FROM documents
WHERE page_count > (
    SELECT AVG(page_count)
    FROM documents
);
```

The inner query is independent.

Correlated

The subquery refers to the outer query.

```
SELECT
u.name

FROM users u
WHERE EXISTS (
    SELECT 1
    FROM documents d
    WHERE d.user_id = u.id
);
```

Here:

u.id

comes from the outer query.

So the inner query is correlated.

21. Correlated Subquery Example

Question:

Find documents whose page count is greater than the average page count of documents owned by the same user.

This is a great example.

```
SELECT
d.id,

d.title,

d.user_id,
```

```
d.page_count  
  
FROM documents d  
WHERE d.page_count > (  
    SELECT AVG(d2.page_count)  
    FROM documents d2  
    WHERE d2.user_id = d.user_id  
);
```

Look carefully.

Outer query:

d

Inner query:

d2

Correlation:

```
WHERE d2.user_id = d.user_id
```

Meaning:

Compare this document against the average document size of its own owner.

22. Why We Need Two Aliases

We have the same table twice:

documents d

and:

documents d2

Think:

d

↓

current document

d2

↓

documents used to calculate comparison average

Without aliases, it becomes ambiguous.

23. RAG Example: Above-User-Average Documents

Suppose user Aarav has:

20 pages

40 pages

60 pages

Average:

40

The query:

```
WHERE d.page_count > (  
    SELECT AVG(d2.page_count)  
    FROM documents d2  
    WHERE d2.user_id = d.user_id  
)
```

returns:

60 pages

For another user, the average is calculated independently.

This is a powerful analytical pattern.

24. Subquery in SELECT

A subquery can also appear inside SELECT.

Example:

```
SELECT  
d.title,  
  
d.page_count,  
  
(  
    SELECT AVG(page_count)  
    FROM documents  
) AS overall_average  
FROM documents d;
```

Result conceptually:

title	page_count	overall_average
RAG Fundamentals	20	35.4
PostgreSQL Guide	45	35.4
Python Basics	30	35.4

The same average appears for every row.

25. Why Would This Be Useful?

You can calculate:

current value

vs

overall benchmark

For example:

document page count

overall average page count

Then perhaps later calculate:

difference

percentage difference

classification

26. Subquery in FROM

A subquery can also behave like a temporary table.

Example:

```
SELECT
  user_id,

  total_pages

FROM (
  SELECT
    user_id,

    SUM(page_count) AS total_pages
  FROM documents
```

```
GROUP BY user_id  
) user_totals;
```

The inner query produces:

user_totals

Conceptually:

documents

↓

```
GROUP BY user
```

↓

user_totals

↓

outer query

This is called a **derived table**.

27. Why Use a Subquery in FROM?

Suppose you first need to calculate:

total pages per user

and then perform another operation on those results.

Example:

Find users whose total pages exceed 100.

```
SELECT  
user_id,  
  
total_pages  
  
FROM (  
  SELECT  
    user_id,  
  
    SUM(page_count) AS total_pages  
  FROM documents  
  GROUP BY user_id  
) user_totals  
WHERE total_pages > 100;
```

The inner query creates a result set.

The outer query filters it.

28. Subquery Execution Mental Model

Don't think of SQL as simply:

line 1

line 2

line 3

Think in logical stages.

For:

```
SELECT
title

FROM documents
WHERE page_count > (
    SELECT AVG(page_count)
    FROM documents
);
```

Conceptually:

1. Calculate AVG(page_count)

↓

2. Get average

↓

3. Scan documents

↓

4. Compare page_count

↓

5. Keep matching rows

↓

6. Return title

This mental model is useful for debugging.

29. Subquery + Aggregation

Example:

Find users whose document count is greater than 1.

Simple:

```
SELECT
  user_id,
  COUNT(*) AS document_count
FROM documents
GROUP BY user_id
HAVING COUNT(*) > 1;
```

Now let's compare each user's count to the **average number of documents per user**.

One approach:

```
SELECT
  user_id,
  COUNT(*) AS document_count
FROM documents
GROUP BY user_id
HAVING COUNT(*) > (
  SELECT AVG(document_count)
  FROM (
    SELECT
      user_id,
      COUNT(*) AS document_count
    FROM documents
    GROUP BY user_id
  ) counts
);
```

This query has three conceptual levels:

documents

↓

count documents per user

↓

calculate average user count

↓

find users above average

This is exactly the kind of problem subqueries help solve.

30. Common Subquery Mistakes

Mistake 1 — Using = with multiple rows

Bad:

```
WHERE user_id = (  
  SELECT id  
  FROM users  
  WHERE country = 'India'  
);
```

If multiple Indian users exist, this fails.

Use:

```
WHERE user_id IN (  
  SELECT id  
  FROM users  
  WHERE country = 'India'  
);
```

Mistake 2 — Forgetting correlation

Suppose you want the average page count **for the same user**.

Incorrect:

```
SELECT  
d.title  
  
FROM documents d  
WHERE d.page_count > (  
  SELECT AVG(page_count)  
  FROM documents  
);
```

This compares against the global average.

For per-user average:

```
SELECT  
d.title  
  
FROM documents d  
WHERE d.page_count > (  
  SELECT AVG(page_count)  
  FROM documents  
  WHERE d.user_id = d.user_id  
);
```

```
SELECT AVG(d2.page_count)
FROM documents d2
WHERE d2.user_id = d.user_id
);
```

Mistake 3 — NOT IN with NULL

Potentially dangerous:

```
WHERE id NOT IN (
  SELECT user_id
  FROM documents
);
```

if the subquery can contain NULL.

Consider:

NOT EXISTS

instead.

Mistake 4 — Returning multiple columns where one value is expected

This is invalid conceptually:

```
WHERE page_count > (
  SELECT
    page_count,
    status
  FROM documents
);
```

The outer comparison expects one value, not two columns.

31. Debugging Challenge #1

This query:

```
SELECT
  title

FROM documents
WHERE user_id = (
  SELECT id
  FROM users
```

```
WHERE country = 'India'  
);
```

Suppose India has multiple users.

Explain:

1. Why can this fail?
2. What operator should be used?
3. What should the corrected structure look like?

Don't just memorize IN.

Explain **why**.

32. Debugging Challenge #2

Developer wants:

Documents larger than the average document belonging to the same user.

They write:

```
SELECT  
d.title,  
  
d.page_count  
  
FROM documents d  
WHERE d.page_count > (  
    SELECT AVG(page_count)  
    FROM documents  
);
```

Is this correct?

If not:

1. What average is being calculated?
2. What does the business requirement actually require?
3. How would you correlate the inner query?

33. Debugging Challenge #3

Developer wants:

Users with no documents.

They write:

```
SELECT
  u.id,
  u.name
FROM users u
WHERE u.id NOT IN (
  SELECT d.user_id
  FROM documents d
);
```

The query sometimes behaves unexpectedly.

What potential SQL issue should you investigate?

What alternative could you use?

34. SQL Exercises

Use:

users

documents

document_chunks

embeddings

Q1

Find documents whose page count is greater than the **average page count of all documents**.

Return:

document_id

title

page_count

Q2

Find documents having the **maximum page count**.

Q3

Find users who have uploaded at least one document using EXISTS.

Return:

user_id

name

Q4

Find users who have **no documents** using NOT EXISTS.

Q5

Find documents owned by users from India using IN.

Q6

Find documents owned by users whose account status is active.

Use a subquery.

Q7

Find documents whose page count is greater than the **average page count of documents owned by the same user**.

This is your first serious correlated subquery.

35. Intermediate Exercises

Q8

Find users whose number of documents is greater than the **average number of documents per user**.

Return:

user_id

document_count

Q9

Find the user(s) who have the **largest total number of document pages**.

Return:

user_id

total_pages

If multiple users tie, return all of them.

Hint:

You'll probably need:

GROUP BY

SUM

MAX

subquery

Q10

Find documents that belong to users who have uploaded **more than 2 documents**.

Return:

document_id

title

user_id

Q11

Find documents whose page count is greater than the **average page count of processed documents**.

Q12

Find users who have at least one document with:

page_count > 50

Use EXISTS.

Q13

Find users who have **no failed documents**.

Think carefully:

"No failed documents" is not necessarily the same as "no documents."

36. Engineering Challenge — RAG Analytics

Your RAG platform wants to identify:

Users who have uploaded documents larger than their own average document size.

Return:

user_id

user_name

document_id

document_title

page_count

user_average_page_count

Requirements:

- Use a correlated subquery.
- Compare each document with its owner's average.
- Don't hard-code any average.

- **Sort by:**

page_count DESC

Conceptual logic:

For each document:

↓

Find its owner's average document size

↓

Compare current document

↓

Keep if current > average

37. Engineering Challenge 2

RAG ingestion team asks:

"Find users who have uploaded more documents than the average user."

Return:

user_id

user_name

document_count

average_user_document_count

You need to think about two aggregation levels:

documents

↓

documents per user

↓

average documents per user

↓

compare each user

This is intentionally challenging.

38. Interview Questions

Answer in your own words.

Q14

What is a subquery?

Q15

What's the difference between:

=

and:

IN

when used with subqueries?

Q16

What is a correlated subquery?

Q17

What's the difference between:

IN

and:

EXISTS

conceptually?

Q18

Why can NOT IN behave unexpectedly when NULL exists?

Q19

When might a JOIN be clearer than a subquery?

Q20

When is EXISTS useful in a RAG system?

Give a practical example.

39. Explanation Challenge

Explain this query line by line:

```
SELECT
d.id,

d.title,

d.page_count

FROM documents d
WHERE d.page_count > (
    SELECT AVG(d2.page_count)
    FROM documents d2
    WHERE d2.user_id = d.user_id
)
ORDER BY d.page_count DESC;
```

Your explanation must cover:

1. What d represents.
2. What d2 represents.
3. Why the documents table appears twice.
4. Why d2.user_id = d.user_id is necessary.
5. What value the inner query returns.
6. What the outer query compares.
7. Why this is a correlated subquery.
8. What happens if a user has only one document.
9. What happens if two documents tie above the average.
10. What ORDER BY does.

40. Data Engineering Thinking

Imagine:

documents = 500 million rows

and you write:

```
SELECT
d.id,

d.page_count
```

```
FROM documents d
WHERE d.page_count > (
  SELECT AVG(d2.page_count)
  FROM documents d2
  WHERE d2.user_id = d.user_id
);
```

Think about:

Question 1

How many times might the correlated subquery logically be evaluated?

Question 2

Why could this be expensive?

Question 3

Could you solve the same problem using:

```
GROUP BY
JOIN
or:
```

CTE

?

Question 4

Would an index on:

documents.user_id

potentially matter?

Don't worry about optimizing it yet.

We're building the mental model first.

41. Short Test

Q21

What does a scalar subquery return?

- A. Multiple tables
- B. One value
- C. A database
- D. Only text

Q22

Which operator is appropriate when a subquery returns multiple values?

- A. =
- B. >
- C. IN
- D. AS

Q23

What does EXISTS check?

- A. Whether a column is numeric
- B. Whether at least one matching row exists
- C. Whether a table is empty
- D. Whether a query is indexed

Q24

What makes a subquery correlated?

- A. It contains GROUP BY
- B. It uses ORDER BY
- C. It references a column from the outer query
- D. It uses COUNT

Q25

What is the potential problem with:

NOT IN (

```
SELECT user_id  
FROM documents  
)
```

if user_id contains NULL?

Q26

Which is comparing against each user's own average?

A

```
WHERE d.page_count > (  
  SELECT AVG(page_count)  
  FROM documents  
);
```

B

```
WHERE d.page_count > (  
  SELECT AVG(d2.page_count)  
  FROM documents d2  
  WHERE d2.user_id = d.user_id  
);
```

Explain why.